Who submitted the workload?

How to reproduce the tests and what is still missing | 16 October 2026

Enable it in the exporter

This adds `submitted_by` to `karta_pod_workload_info` and `karta_workload_status`. Karta's CRD and SDK stay unchanged. Admission captures direct Kubernetes submissions. CEL reads the metadata supplied by each platform.

```
exporter:
  enabled: true
  useCatalog: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

Admission records the authenticated Kubernetes caller before a selected workload is saved. Karta's operator serves this webhook over HTTPS (TLS). Within the selected scope, it also rejects changes to the recorded identity. Keep `operator.enabled` and `webhook.enabled` true. Setting `exporter.attribution.admission.enabled: false` stops capture; CEL can still read existing platform metadata.

Choose namespaces and workload types

Already supported: namespace include/exclude selectors and a resource allowlist. Add this under the existing exporter values; keep `submittedByExpression` unchanged.

```
exporter:
  attribution:
    admission:
      enabled: true
      namespaceSelector:
        matchExpressions:
          - key: karta.run.ai/capture-submitter
            operator: In
            values: ["true"]
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values: [kube-system, karta-system]
      resourceRules:
        - apiGroups: ["apps"]
          apiVersions: ["v1"]
          resources: ["deployments", "statefulsets"]
        - apiGroups: ["batch"]
          apiVersions: ["v1"]
          resources: ["jobs", "cronjobs"]
        - apiGroups: [""]
          apiVersions: ["v1"]
          resources: ["pods"]
```

Include namespaces labeled `karta.run.ai/capture-submitter=true`; exclude the two named namespaces even if labeled. Both expressions must match. To include namespace names, use `kubernetes.io/metadata.name` with `In`.

Resource rules use API group/version and plural resource names: `apps/v1/deployments` selects Deployment. Today, exclude a type by removing it from this allowlist. Replace `karta-system` with the operator's namespace.

Proposed: match group, version and kind (GVK)

The chart does not expose `matchConditions` yet. Proposed addition under the same admission block, applied to both capture and update protection:

```
exporter:
  attribution:
    admission:
      matchConditions: # Proposed; not implemented in the chart
        - name: include-gvks
          expression: |
            request.kind.version == "v1" && (
              (request.kind.group == "apps" &&
               request.kind.kind in ["Deployment", "StatefulSet"]) ||
              (request.kind.group == "batch" &&
               request.kind.kind in ["Job", "CronJob"]) ||
              (request.kind.group == "" && request.kind.kind == "Pod"))
        - name: exclude-gvks
          expression: |
            !(request.kind.group == "batch" &&
              request.kind.version == "v1" &&
              request.kind.kind == "CronJob")
```

All conditions must pass, so the CronJob exclusion wins. Excluded requests skip submitter capture and update protection. `submittedByExpression` separately selects the identity to export. Namespace labels and scope settings must remain administrator-controlled.

This follows Kubernetes webhook matching. Only `matchConditions` is proposed; namespace selectors and resource allowlists already work.

Direct Kubernetes submission

Download the Job and Deployment request files into your working directory.

Create and label the test namespace as an administrator:

```
kubectl create namespace karta-label-direct
kubectl label namespace karta-label-direct \
  karta.run.ai/capture-submitter=true
```

Use a user's kubeconfig with permission to create/patch Jobs and Deployments in that namespace. The test used a one-hour client certificate for `alice@example.test`, without impersonation.

```
kubectl --kubeconfig "$USER_KUBECONFIG" auth whoami
kubectl --kubeconfig "$USER_KUBECONFIG" apply \
  -f "direct-job-request.json"
kubectl --kubeconfig "$USER_KUBECONFIG" apply \
  -f "direct-deployment-request.json"
```

The Deployment ran; the Job completed after 300 seconds. Karta admission wrote the caller onto each submitted object, not a Karta CR. Captured Job fields:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cel-label-job
  annotations:
    workload.dgxc.nvidia.com/submitter-id: alice@example.test
```

CEL reads `annotations["workload.dgxc.nvidia.com/submitter-id"]`. The Pod metric inherits this value through workload attribution; its Pod has no copied annotation. `ownerReferences` links controllers and children; it does not identify the human.

```
karta_workload_status{
  workload="cel-label-job", phase="Completed",
  submitted_by="alice@example.test"
} 1
karta_pod_workload_info{
  workload="cel-label-job", submitted_by="alice@example.test"
} 1
```

Try supplying another identity at creation, then changing the recorded identity:

```
KEY=workload.dgxc.nvidia.com/submitter-id
jq --arg key "$KEY" '
  .metadata.name="cel-label-forged" |
  .metadata.annotations[$key]="bob@example.test"
' "direct-job-request.json" |
  kubectl --kubeconfig "$USER_KUBECONFIG" create -f -

kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-label-direct \
  annotate job cel-label-job "$KEY=bob@example.test" --overwrite
```

Both were rejected: creation conflicted with the authenticated caller; the update changed an immutable field. Prefilling the caller's own matching identity is allowed. Unrelated labels and annotations are preserved.

This protection covers the selected resources and namespaces. It does not authenticate arbitrary platform metadata. System-created owned children are skipped, and a Kubernetes service account does not reveal the human behind it.

Direct submission and rejection evidence includes the original requests and both rejection results.

Run:AI

Download the training and inference request files into your working directory.

Set `RUNAI_API`, `RUNAI_TOKEN`, `PROJECT_ID` and `CLUSTER_ID`. The test logged in through `/api/v1/token` using a password grant. Use a token with permission to submit into your project.

```
for kind in training inference; do
  jq --arg p "$PROJECT_ID" --arg c "$CLUSTER_ID" \
    '.projectId=$p | .clusterId=$c' \
    "runai-$kind-request-template.json" > "$kind.json"
done

curl -fsS "$RUNAI_API/api/v1/workloads/trainings" \
  -H "Authorization: Bearer $RUNAI_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @"training.json"
curl -fsS "$RUNAI_API/api/v1/workloads/inferences" \
  -H "Authorization: Bearer $RUNAI_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @"inference.json"
```

Both use BusyBox, 50m CPU and 64Mi memory, without GPUs. Training completed after 300 seconds. Inference served HTTP 200 on port 8080 with one replica.

Run:AI supplied the identity on these objects. Captured fields:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cel-label-training
  annotations:
    run.ai/submitted-by: test@run.ai
---
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: cel-label-inference
  annotations:
    run.ai/submitted-by: test@run.ai
```

CEL reads `annotations["run.ai/submitted-by"]`. The WorkloadRunner CRs had no submitter annotation. Karta attributed these Pods to the Job and Service above. The inference Pod had no submitter key; its metric inherited the Service's value.

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
karta_pod_workload_info{
  workload="cel-label-inference", submitted_by="test@run.ai"
} 1
```

Extraction is generic. Knative was the runtime of this inference test; the same CEL reads the annotation on the training Job.

- Runtime setup: Knative's `kubernetes.podspec-schedulername` and `kubernetes.podspec-affinity` flags allowed this test's scheduler and affinity settings. They do not enable submitter extraction.
- Exporter access: `get/list/watch` on `run.ai/workloadrunners` lets Karta read the Run:AI ownership graph. The separate exporter was for lab isolation. No identity mapper was used.
- Test limit: HTTP was checked through a Knative revision proxy port-forward. External ingress and model inference were not tested.

Current gap: agree the NVIDIA username format with Run:AI. No submitter propagation bug was observed in these two paths.

See the captured resource fields and Run:AI metrics.

OSMO

Use a logged-in OSMO CLI. Replace `YOUR_CPU_PLATFORM` and `YOUR_POOL` with your platform and pool. Each block creates and submits the complete workflow; no files to download.

Single task:

```
cat > osmo-single.yaml <<'YAML'
workflow:
  name: cel-label-single
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  tasks:
  - name: worker
    image: ubuntu:24.04
    command: [/bin/sh, -c]
    args:
    - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-single.yaml --pool YOUR_POOL
```

Leader/worker group:

```
cat > osmo-group.yaml <<'YAML'
workflow:
  name: cel-label-group
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  groups:
  - name: pair
    tasks:
    - name: leader
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
      lead: true
    - name: worker
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-group.yaml --pool YOUR_POOL
```

Use the workflow ID returned by each command to check completion:

```
osmo workflow query YOUR_WORKFLOW_ID
```

These are the same workflow specifications used in the API test below. The CLI commands are the documented reproduction path; the saved results came from REST submissions.

OSMO: result and gaps

The requests submit one task and a leader/worker group. Each task uses Ubuntu 24.04, one CPU, 512Mi memory and a 300-second command. Both workflows completed; all three tasks in the repeated run exited with code 0.

The workflow API returned `submitted_by: alice@example.test`. Kubernetes had no workflow CR in this setup. The identity available to CEL was on the task Pod. Captured fields:

```
apiVersion: v1
kind: Pod
metadata:
  name: e5d1abe4ce374717-e923b371aa294f01
  labels:
    osmo.workflow_id: cel-label-single-2
    osmo.task_name: worker
    osmo.submitted_by: alice
```

CEL reads `labels["osmo.submitted_by"]`. Karta used each task Pod as the workload; the full identity in the OSMO API was not read by the exporter.

```
karta_pod_workload_info{
  workload="e5d1abe4ce374717-e923b371aa294f01",
  workload_kind="Pod", submitted_by="alice"
} 1
```

`karta_workload_status` carried the same `submitted_by="alice"`. Samples were captured while the tasks were running.

The API test used OSMO 6.3.1 with development headers `x-osmo-user: alice@example.test` and `x-osmo-roles: osmo-user`. Those headers supplied the test identity; production login was not tested. The workflows ran on a separate platform using the existing scheduler. No external identity mapper was used.

Current OSMO bugs: the label drops the email domain, and the one-character user `a` produced no submitter label. Different emails can become the same `alice`.

For now, treat `alice` as a shortened name, not a unique user. Recovering the full identity requires a workflow API lookup, which this PoC does not implement. Suggested OSMO issue: preserve the full principal in an annotation and fix the one-character case.

Current Karta/OSMO gap: there is no Kubernetes workflow root in this setup. Today the metric represents each task Pod. Suggested integration issue: provide a workload-root mapping so one workflow can contain all its tasks.

OSMO outcomes show completion and task exit codes. The original identity edge tests cover the shortened and missing values.

NVCF / NVCT

NVCF ran an HTTP service; NVCT ran a task that finishes. Both worked. Neither exposed the human submitter to Kubernetes, so `submitted_by` stayed empty.

1. NVCF HTTP service: download the function and deployment requests. Use your management API URL/token for `NVCF_API` and `NVCF_ADMIN_TOKEN`. Match the deployment's `gpu`/`instanceType` to your compute pool.

```
curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @nvcf-function-request.json > function.json
FID=$(jq -r '.function.id' function.json)
VID=$(jq -r '.function.versionId' function.json)
DEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$FID/versions/$VID"
curl -fsS "$DEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @nvcf-deploy-request.json
```

Once ACTIVE, call it with your gateway URL/domain and invocation key. The lab gateway routes by the function's Host header:

```
curl -fsS "$NVCF_INVOKE_URL/" \
  -H "Host: $FID.$NVCF_INVOKE_DOMAIN" \
  -H "Authorization: Bearer $NVCF_API_KEY"
```

Observed: the Nginx service returned HTTP 200.

2. NVCT finite task: download the task request and match its `gpu`/`instanceType` to the pool. Set your task API URL/key. Undeploy the function to free its allocation:

```
curl -fsS -X DELETE "$DEPLOY" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS "$NVCT_API/v1/nvct/tasks" \
  -H "Authorization: Bearer $NVCT_API_KEY" \
  -H 'Content-Type: application/json' --data-binary @nvct-task-request.json
```

Observed: COMPLETED at 100%, with result uploads disabled.

NVCF / NVCT: where the identity is missing

The Kubernetes CR was `ICMSRequest` (`nvca.nvcf.nvidia.io/v2beta1`). Its metadata had account and function/task IDs, but no human submitter field. The Pods also lacked that field.

These are the observed label paths, not user identities:

```
ICMSRequest metadata.labels:
  nca-id
  nvca.nvcf.nvidia.io/function-id          # NVCF
  nvca.nvcf.nvidia.io/function-version-id  # NVCF
  nvca.nvcf.nvidia.io/task-id              # NVCT
Pod metadata.labels:
  nca-id
  function-id, function-version-id        # NVCF
  task-id                                 # NVCT
```

Both existing metric families had `submitted_by=""`. Captured HTTP-service sample:

```
karta_pod_workload_info{
  workload="0-sr-df24444a-8807-4fc9-89c6-534c7a701643",
  workload_kind="Pod", submitted_by=""
} 1
```

To expose the human submitter, NVCF needs to capture a verified caller at submission and pass it to NVCA, the cluster agent. NVCA would write it onto the ICMSRequest and its Pods. For functions, first agree whether this means creator or deployer; the caller invoking a running function is a different identity.

Proposed metadata on both objects, not present in the captures:

```
metadata:
  annotations:
    workload.dgxc.nvidia.com/submitter-id: alice@example.test
```

The CEL expression on page 1 already reads this key. Stamping only the CR would not cover this test: Pod owner links lacked `controller: true`, so Karta treated each Pod as the workload. Function/task-level grouping also needs a Karta/NVCF integration fix.

For now, leave the value empty. An external lookup could correlate account and function/version/task IDs with an audit record, but only if it contains the original user. No lookup was implemented. A service credential alone cannot identify a human. Suggested NVCF issue: propagate the authenticated submitter to Kubernetes metadata.

Other current platform gaps: NVCT helper image references did not resolve; overrides `nvcf-worker-init-oss:1.2.1` and `nvcf-worker-task-oss:1.1.4` worked. NVCA left an ICMSRequest after malformed-task cancellation; manual cleanup was required. Suggest issues for both.

Lab setup: self-managed NVCF with service credentials from OpenBao; test-container CPU reduced to 100m. Hosted NVCF and human login were not tested.

See the NVCF capture, NVCT capture and identity issue draft. No issues have been opened.